

Impacto de las Épocas Offline en el Rendimiento y la Exactitud de Entrenamientos Distribuidos con O.N.O.

Marcos Bianchi

Facultad de Ingeniería, Universidad de Buenos Aires

Aprendizaje Profundo

Julio 2026

Resumen—Retrasar la sincronización entre nodos en un entrenamiento distribuido reduce el impacto de la red pero induce divergencia entre los pesos (*client drift*). Este trabajo evalúa ese impacto de variar las épocas offline (E) utilizando el sistema de entrenamiento O.N.O. empleando *Parameter Server* sincrónico y *All-Reduce* (*ring*). Entrenando LeNet-5 sobre MNIST FULL en un cluster simulado mediante Docker en una sola computadora, los resultados muestran que incrementar E no reduce el tiempo total debido a la contención del CPU, mientras que degrada la *accuracy* final e incrementa la inestabilidad del entrenamiento.

Palabras clave—Épocas Offline, *Parameter Server*, *All-Reduce*, O.N.O., *Client Drift*.

I. INTRODUCCIÓN Y OBJETIVOS

Permitir que los nodos operen de forma autónoma durante algunas épocas posterga el intercambio de parámetros, disminuyendo la dependencia de la red y priorizando el cómputo local. Sin embargo, demorar la sincronización introduce divergencia en los pesos de los nodos, un fenómeno conocido como *client drift* [1].

Pregunta de investigación. ¿Cómo impactan las épocas offline a la velocidad y eficacia del modelo entrenado?

Objetivos. Evaluar esta pregunta utilizando O.N.O., un sistema de entrenamiento distribuido de modelos de aprendizaje profundo implementado como parte de un trabajo de fin de carrera. Con ello ejecutar varios entrenamientos sobre una misma arquitectura, dataset e hiperparámetros para relevar las diferencias en cuanto al impacto real de las épocas offline.

II. ARQUITECTURA DEL SISTEMA Y MECANISMOS DE SINCRONIZACIÓN

II-A. Componentes Principales de O.N.O.

- **Orchestrator:** Mediante una configuración declarativa del entrenamiento brindada por el usuario, configura al resto de nodos para iniciar el entrenamiento de un modelo usando alguno de los algoritmos implementados. Al finalizar un entrenamiento, consolida los resultados parciales generando un archivo *safetensors* con el resultado final que contiene los parámetros del modelo entrenado. Para una mejor automatización, cuenta con una interfaz FFI de Python que fue utilizada para simular los ensayos discutidos más adelante.
- **Workers:** Computan gradientes locales por backpropagation sobre una partición asignada del dataset. En All-

Reduce aplican localmente el optimizador mientras que en *Parameter server* delegan la actualización a los servidores.

- **Servers:** Nodos dedicados al guardado y optimización de parámetros del modelo entrenado. Reciben gradientes parciales de los workers, los agregan y aplican sobre los parámetros para luego redistribuirlos y volver a empezar.

II-B. Dinámica de Sincronización Demorada ($E > 1$)

Al parametrizar $E > 0$, los nodos entrenan aislados durante múltiples ciclos. El tráfico de red se reduce considerablemente de forma inversamente proporcional a E . No obstante, la falta de sincronización de estado causa que los pesos locales de cada uno de los workers diverjan (*client drift*). Al alcanzar la etapa de sincronización, la consolidación se realiza por el promedio directo de los gradientes parciales.

Bajo un entrenamiento optimizado por *Descenso por Gradiente*, una ecuación que describe la etapa de optimización es:

$$W_{t+1} = W_t - \frac{\lambda}{N} \sum_{i=1}^N G_i \quad (1)$$

II-C. Algoritmos de Entrenamiento Distribuido

El sistema implementa varios algoritmos de entrenamiento distribuido, en particular el informe se va a centrar en los siguientes:

- **Parameter Server Sincrónico:** Un algoritmo centralizado bajo el modelo propuesto por Li et al. [2] donde una barrera bloquea a los *workers* hasta que cada *server* procesa el lote global de gradientes, garantizando que todos los *workers* estén en todo momento calculando gradientes de la misma epoch.
- **All-Reduce:** Un algoritmo descentralizado inspirado en arquitecturas de anillo de alto rendimiento como Horovod [3], donde los *workers* se organizan en una topología de anillo lógico compartiendo fragmentos de gradientes en fases sucesivas de scatter y gather, calculando de forma idéntica y local el vector de parámetros resultante.

III. METODOLOGÍA EXPERIMENTAL

Todos los entrenamientos fueron realizados en la misma computadora de forma secuencial utilizando Docker para simular un entorno multicomputadora, sumando un total de

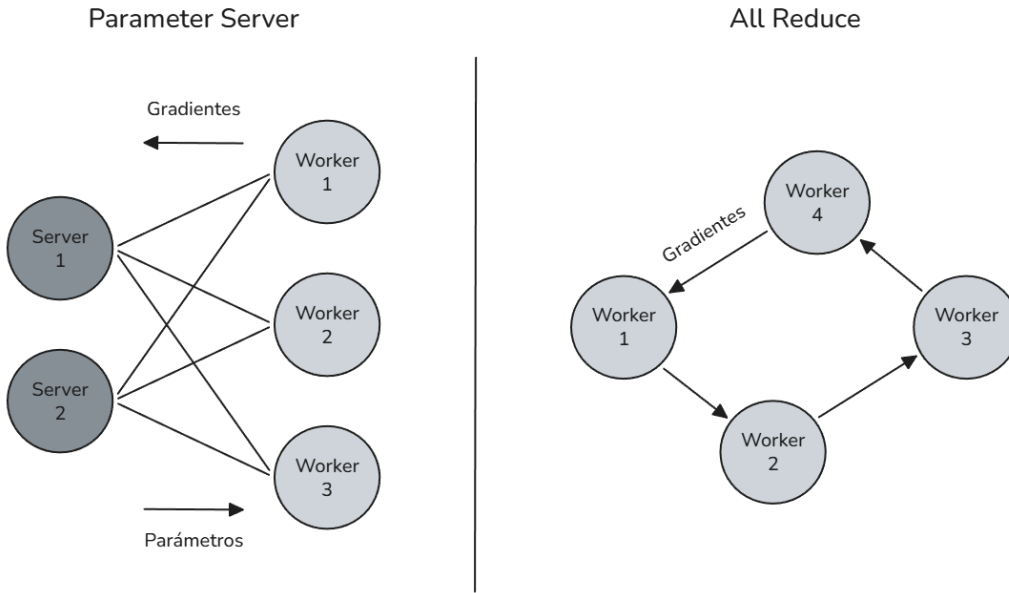


Figura 1: Topologías distribuidas soportadas en O.N.O. (izquierda) Arquitectura centralizada bajo *Parameter server* donde los *workers* obtienen parámetros y envían gradientes a *servers* fragmentados. (derecha) Topología descentralizada en anillo *All-Reduce* donde cada nodo actúan como *worker* comunicándose con sus nodos adyacentes.

10,50 horas de ejecución. Las especificaciones técnicas del entorno se detallan en la Tabla II, mientras que las variables lógicas y configuraciones del modelo O.N.O. se consolidan en la Tabla I.

Tabla I: Configuración de Ensayos

Categoría	Parámetro	Valores Evaluados
Modelo y Datos	Dataset	MNIST (Full) [4]
	Arquitectura de Red	LeNet-5
	Función de Pérdida	Entropía Cruzada
Optimización	Optimizador	Adam
		$\alpha = 0,01$
		$\beta_1 = 0,9$
		$\beta_2 = 0,999$
		$\epsilon = 10^{-8}$
	Tamaño de Mini-Batch	64
	Épocas Máximas	48
Infraestructura	Serialización	Sparse ($r = 0,5$)
Variables Libres	Algoritmo Distribuido	All-Reduce, Parameter Server (Sincrónico/Bloqueante)
	Nodos	2, 4, 6, 8, 10
	Épocas Offline	0, 1, 2, 3, 4

Para el algoritmo de *Parameter Server*, el total de nodos se compone de forma que la mitad de ellos son *workers* y la otra *servers*.

III-A. Compresión de Red: Serialización Sparse

La configuración de la sección de infraestructura responde al método de serialización de gradientes del sistema, el cual soporta una variante *sparse* (o dispersa) diseñada para optimizar la comunicación distribuida limitando el intercambio de sectores poco impactantes de los gradientes [5]. El parámetro

r sirve para acotar el sampleo del gradiente que se quiere transmitir: a mayor r , mayor será la compresión teórica.

El algoritmo calcula la cardinalidad efectiva del mensaje mediante:

$$k = \text{round}(|g| \cdot (1,0 - r)) \quad (2)$$

El valor resultante k se utiliza para escoger los k elementos de mayor magnitud absoluta dentro del tensor de gradientes local, fijando un umbral mínimo. Todo valor que alcance o supere dicho umbral se incorporará al mensaje por enviar. De esta forma, un valor de $r = 0,0$ samplea la totalidad del gradiente, mientras que un $r = 0,5$ (como el configurado para estos experimentos) filtra el 50,00 % de los componentes para disminuir el transito por red. Los elementos descartados se acumulan en un gradiente residual para ser considerados en el próximo mensaje.

Tabla II: Entorno de Ejecución

Categoría	Componente	Detalle
Procesador	Modelo	Intel Core i5-8350U
	Arquitectura	x86_64
	Núcleos Físicos	4
	Hilos Lógicos	8
	Frecuencia de Reloj	Base: 1,70 GHz Turbo: 3,60 GHz
Memoria	Capacidad Total RAM	8 GB
	Tipo	DDR4
	Velocidad	2400,00 MHz
Entorno de Software	Sistema Operativo	Ubuntu 24.04 LTS
	Entorno de Ejecución	Docker v29.6.1
		Rust v1.96.1
		CPython v3.14.0

IV. RESULTADOS Y ANÁLISIS EXPERIMENTAL

IV-A. Análisis Temporal y Escalabilidad del Hardware

La variación de E no indujo mejoras significativas en los tiempos de ejecución de los entrenamientos; las curvas de cada configuración exhiben un solapamiento estricto en todos los ensayos. Dado que el sistema se ejecuta en una única máquina, la supresión del costo de comunicación no logra compensar la sobrecarga de cómputo, contrariamente a las expectativas iniciales de optimización.

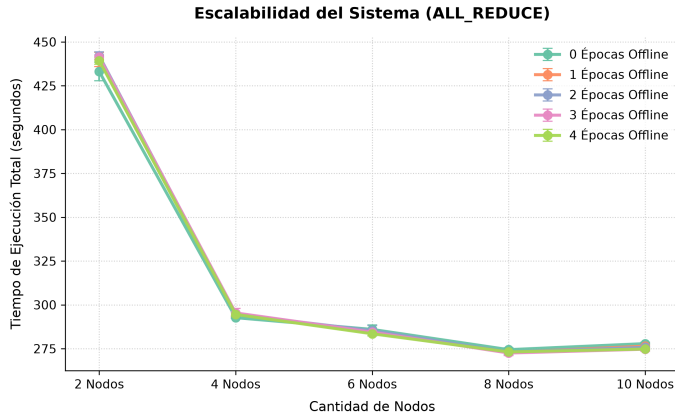


Figura 2: Comparación de tiempos de ejecución bajo distintas cantidades de Épocas Offline con All-Reduce.

Como se observa en la Figura 2, la escalabilidad del rendimiento se estanca al superar los 4 nodos. En *All-Reduce*, la elevación a 10 nodos degrada el desempeño global debido a la alta contención por el CPU.

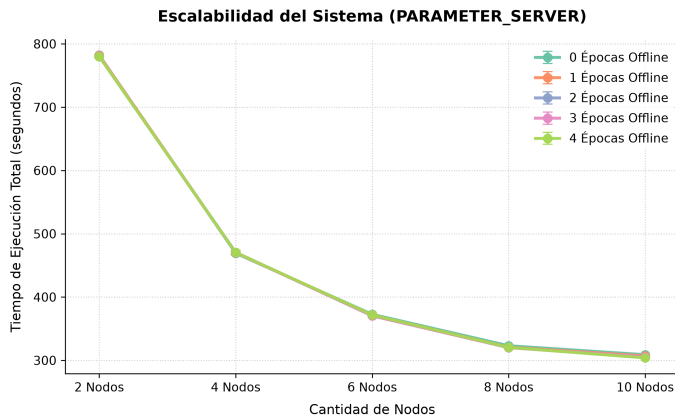


Figura 3: Comparación de tiempos de ejecución bajo distintas cantidades de Épocas Offline con Parameter Server.

En la Figura 3 se constata que la arquitectura de *Parameter Server* con 10 nodos (5 *workers* y 5 *servers*) reduce el tiempo respecto a configuraciones menores. A diferencia de *All-Reduce*, donde los procesos operan en cómputo continuo, el esquema sincrónico de *Parameter Server* introduce estados de inactividad obligatorios mediante barreras mientras el servidor actualiza

el modelo. Este comportamiento disminuye la contención, permitiendo una alternancia eficiente de hilos de ejecución.

Cabe destacar que *All-Reduce* resulta intrínsecamente más veloz en configuraciones de pocos nodos. Esto responde a una distribución balanceada de responsabilidades donde cada entidad computa, transmite u optimiza parámetros de forma sostenida sin dependencias centralizadas.

IV-B. Métricas de Exactitud

El impacto de la variación de E se manifiesta con mayor claridad en las métricas de exactitud (*accuracy*), al ser variables independientes de las limitaciones del hardware de ejecución.

En ambos algoritmos, la exactitud final del modelo decrece de manera monótona a medida que se incrementa E . Este comportamiento es consistente con la teoría: la introducción de una única época offline ($E = 1$) reduce la frecuencia de sincronización global a la mitad, provocando un deterioro inmediato de aproximadamente el 1,00 % en la métrica final.

En un entorno multi-computadora real, un valor mayor de E podría justificar esta pérdida si la aceleración en tiempo permitiera ejecutar un mayor número de épocas totales. Resta determinar si dicho incremento en volumen de entrenamiento compensaría la divergencia algorítmica.

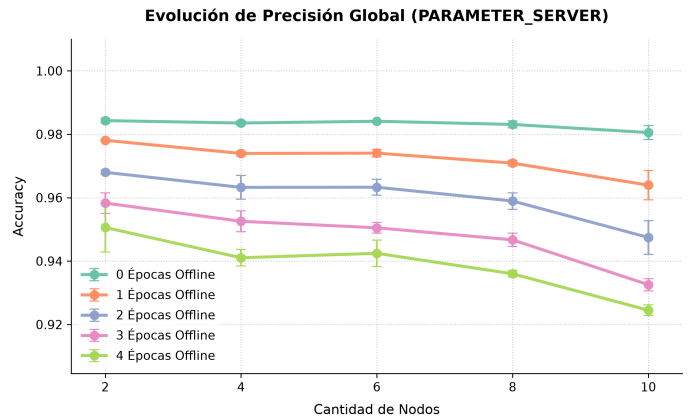


Figura 4: Comparación de accuracy bajo distintas cantidades de Épocas Offline con Parameter Server.

La degradación de la exactitud se acentúa de forma proporcional al número de nodos. Al incrementar las entidades distribuidas, las particiones locales del dataset se reducen y se vuelven potencialmente sesgadas. La falta de sincronización frecuente exacerba el impacto de estos sesgos locales en los *workers*, perjudicando la convergencia global del sistema.

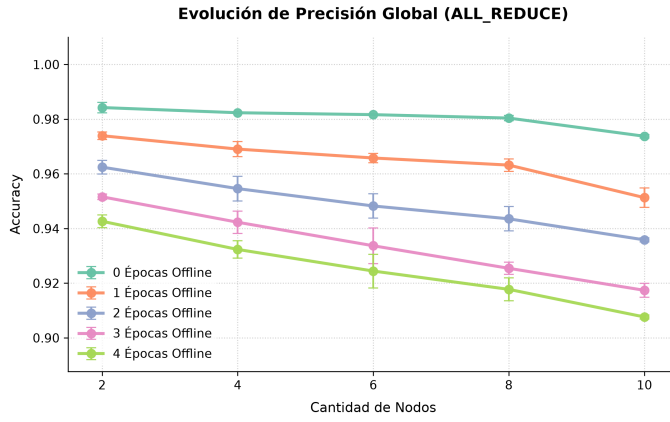


Figura 5: Comparación de accuracy bajo distintas cantidades de Épocas Offline con *All-Reduce*.

En *All-Reduce*, la pérdida de exactitud respecto a la cantidad de nodos y E es más severa que en *Parameter Server*. Bajo las condiciones experimentales, las pruebas de *All-Reduce* duplican el número de *workers* activos al prescindir de servidores dedicados. Esto fragmenta aún más el conjunto de datos e incrementa el aislamiento operativo. La integración tardía de parámetros fuerza correcciones abruptas en el gradiente global, desestabilizando el proceso de optimización.

IV-C. Inestabilidad de Pérdida durante el Entrenamiento

A partir del historial de entrenamiento se seleccionaron trayectorias representativas que confirman que el incremento de E introduce inestabilidad en la función de pérdida. En configuraciones de 2 nodos para ambos algoritmos, esta inestabilidad se evidencia mediante oscilaciones y picos de pérdida proporcionales al valor de E .

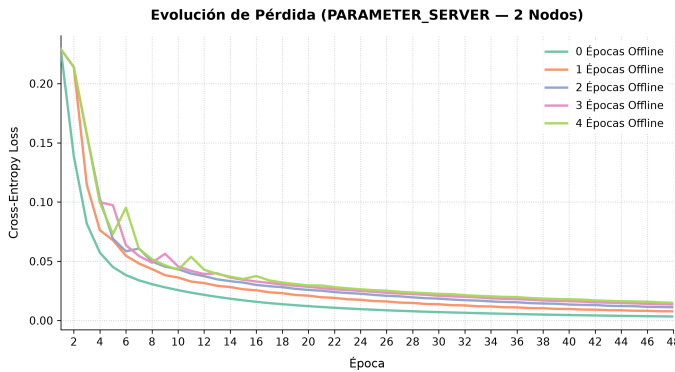


Figura 6: Trayectoria de Loss en Entropía Cruzada exhibiendo ruido estocástico por cómputo offline en *Parameter Server* (2 nodos).

La dinámica del optimizador presenta mayores dificultades en las épocas iniciales, fase asociada a las correcciones de mayor magnitud. Posteriormente, el sistema se estabiliza al aproximarse a un mínimo local. Sin embargo, los valores de convergencia residual son sistemáticamente más altos a mayor E . Este fenómeno sugiere que el desacoplamiento prolongado

restringe la capacidad del optimizador para alcanzar el mínimo global, induciendo un comportamiento análogo al de una tasa de aprendizaje (*learning rate*) excesivamente alta que provoca oscilaciones permanentes.

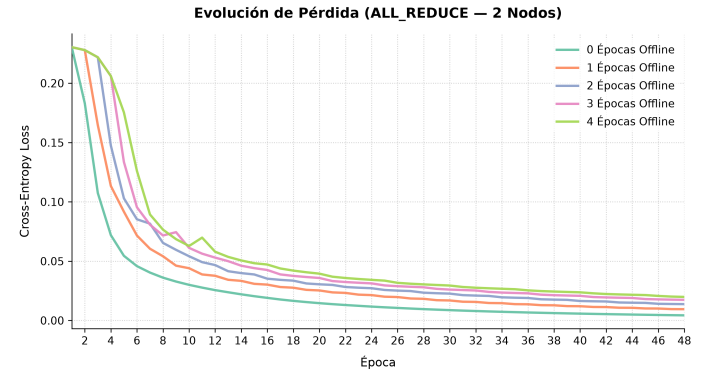


Figura 7: Trayectoria de Loss en Entropía Cruzada exhibiendo ruido estocástico por cómputo offline en *All-Reduce* (2 nodos).

En *All-Reduce* se observa un patrón similar, aunque con oscilaciones relativamente más atenuadas que en *Parameter Server*, revirtiendo la hipótesis inicial que preveía una mayor inestabilidad en el esquema descentralizado.

Considerando que la desestabilización afecta negativamente el resultado, resulta de interés evaluar estrategias de activación dinámica para las épocas offline. Una hipótesis plantea su introducción exclusiva en fases avanzadas del entrenamiento, cuando la optimización principal ha concluido, permitiendo que la exploración independiente de un *worker* en un mínimo local guíe favorablemente al resto de los nodos tras la sincronización.

V. DISCUSIÓN Y LIMITACIONES

Las ventajas de introducir épocas offline es estrictamente bajar el tiempo del entrenamiento ejecutado. En un entorno local simulado, eliminar ese overhead de comunicación no compensa al costo que resulta de un peor modelo. Por lo menos en estos experimentos no se evidenciaron ventajas de utilizar un E mayor a 0.

En contraposición, en un setup multi-computadora sobre una red física, retrasar la agregación de gradientes y las sincronizaciones de nodos puede resultar crítico para amortizar el tiempo de comunicación. Al final del día se tendrá que tomar en cuenta el tamaño del modelo, la velocidad de la red y cuantos nodos se están utilizando.

Es importante notar que con modelos más grandes, los mensajes (que contienen gradientes y parámetros) van a ser grandes también. Para entrenamientos de mucho dataset y poco modelo, el cuello de botella va a ser el cómputo de gradientes y optimización del modelo. En cambio, entrenamientos de poco dataset y mucho modelo, donde la comunicación toma más protagonismo, incrementar E puede ser útil para bajar los tiempos de ejecución y realizar más épocas por segundo.

VI. CONCLUSIÓN

El uso de épocas offline en O.N.O. altera el balance entre comunicación y fidelidad algorítmica en el entrenamiento. En la infraestructura simulada, incrementar E perjudicó al modelo entrenado de manera predecible pero sin aportar ganancias de velocidad, debido a limitaciones del hardware empleado.

Como trabajo futuro, se plantea validar el sistema bajo configuraciones multi-computadora reales, comparando distintas arquitecturas de modelos de distintos tamaños variando E .

REFERENCIAS

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.
- [2] M. Li, L. Zhou, Z. Yang, A. Li, F. Xia, D. G. Andersen, and A. Smola, "Parameter server for distributed machine learning," in *NIPS Workshop on Big Learning*, Lake Tahoe, NV, USA, 2013. [Online]. Available: <https://www.istc-cc.cmu.edu/publications/papers/2013/ps.pdf>
- [3] A. Sergeev and M. Del Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," *arXiv preprint arXiv:1802.05799*, 2018.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, September 2017, pp. 440–445. [Online]. Available: <https://aclanthology.org/D17-1045>